

Deep Learning for American Sign Language Gesture Classification

Erin Siedlecki (ets6dq), Natalie Seah (cva3zw), Isabela Barton (rpx2et)

Motivation

American Sign Language (ASL) is the primary mode of communication for an estimated 250,000 to 500,000 deaf and hard-of-hearing individuals in the United States (James et al., 2021). Despite its widespread use, the vast majority of the hearing population has no knowledge of how to communicate with or interpret ASL, creating a significant communication barrier in everyday interactions such as medical appointments, classrooms, workplaces, and other public services. Human interpreters, while ideal, are not always available or financially feasible (Zulu et al., 2017). An automated ASL recognition system could reduce dependence on human interpreters, lower communication costs, and make everyday interactions more accessible for the deaf and hard-of-hearing community.

This project proposes an application of deep learning to bridge this communication gap through automated ASL recognition from static hand gesture images. Specifically, we frame the problem as a multi-class image classification task: given an RGB image of a hand gesture, the model must predict the corresponding ASL letter (A-Z) or digit (0-9). This is a real-world application with meaningful social impact, as a well-performing model could serve as the foundation for a real-time ASL translation tool, an educational app for learning sign language, or an accessibility feature in communication platforms.

This task is well-suited for deep learning because CNNs are designed to detect the visual patterns that distinguish hand gestures, such as finger positions, hand shape, and orientation.

Datasets

We used ASL-HG, a comprehensive American Sign Language hand gesture image dataset introduced by Pranto et al. (2026) and published in *Data in Brief*, available at <https://doi.org/10.1016/j.dib.2026.112492>. ASL-HG contains 36,000 RGB JPG images across 36 classes, covering the full English alphabet (A-Z) and digits (0-9). Each image has a resolution of 300x300 pixels, but prior to training, all images were resized to fixed input dimensions depending on the model architecture. Images were collected from 10 student volunteers in Mirpur, Dhaka, Bangladesh, with each participant contributing 100 samples per class. Data were captured using smartphone HD cameras under varied natural indoor and outdoor lighting conditions, with diversity in backgrounds, viewing angles, hand shapes, and skin tones. The dataset comes in two forms: raw images that preserve natural background variability, and a MediaPipe-processed version in which hand regions have been segmented from the background. We used the raw image version, which retains natural variation in lighting, backgrounds, and viewing angles, and applied our own preprocessing and augmentation pipeline to prepare the data for model training.

A notable limitation of this dataset is that it only contains 10 volunteers, which constrains the diversity of hand appearances available for training and evaluation. This also means our test set is only made up of two volunteers, which limits the statistical strength of our generalization estimates. Despite this constraint, the dataset's real-world variability in lighting, backgrounds, and angles makes it preferable to highly controlled benchmarks for evaluating practical performance.

Related Work

Prior work on ASL gesture recognition has explored a range of architectures and datasets, collectively establishing both strong baselines and important limitations that influenced our approach.

Alsharif et al. (2023) conducted a systematic comparison of five pretrained models (AlexNet, ConvNeXt, EfficientNet, ResNet-50, and VisionTransformer) on a dataset of 87,000 static ASL alphabet images, finding that CNN-based architectures consistently outperformed the transformer-based approach, with ResNet-50 reaching 99.98% accuracy and VisionTransformer reaching a lower 88.59% because of information loss from required input resizing. While near-perfect accuracy figures like these seem impressive, we were skeptical of their real-world meaning. Early in our own project, a random train/test split produced similarly inflated results, which we traced to data leakage between volunteers, meaning the model had memorized subject-specific features rather than learning the signs themselves. Switching to a volunteer-stratified split brought our best model to 84.36%, a result we consider more honest and more meaningful for evaluating real-world generalizability. Kasapbaşı et al. (2022) further demonstrated that dataset quality matters as much as model choice, with their DeepASLR system achieving 99.38% accuracy on a deliberately challenging dataset collected under variable lighting and capture distances. This reinforces the importance of real-world image diversity over controlled benchmark conditions and supports our choice of ASL-HG's raw image version, which similarly captures gestures across varied backgrounds and lighting rather than relying on pre-segmented images that may not reflect deployment environments.

Technical Approach

The goal of this multi-class classification project is to develop a deep learning model capable of accurately classifying ASL gestures from a given image. The inputs for our first two models, the baseline CNN trained from scratch and EfficientNet-B0, consist of 150×150 pixel RGB images, giving an input tensor of shape (3, 150, 150). For ResNet-18, we used the standard input size of 224×224 pixels to conform with that architecture's design. The output of all models is a predicted class label from 36 possible classes (A–Z and 0–9), derived from a probability distribution over all classes using softmax. Across all three models, we used cross-entropy loss and the Adam optimizer.

Our first model is a baseline CNN created from scratch, meaning we did not use any pretrained weights. This model contains four convolutional blocks, each with two convolutional layers. After each block, max pooling reduces spatial dimensions to retain the most prominent features, followed by dropout (0.25) for regularization. Batch normalization is applied after each convolutional layer to speed up training and stabilize the model, and ReLU activations allow the model to learn the complex, non-linear patterns found in ASL hand gestures. Global average pooling then reduces the feature maps to a 256-dimensional vector. The first fully-connected layer maps this to 128 dimensions with a higher dropout rate of 0.50 to further combat overfitting; the second outputs a 36-dimensional vector with logits for each ASL class.

Our second model applies transfer learning using EfficientNet-B0 with pretrained ImageNet weights. This architecture uses compound scaling by applying fixed ratios to jointly scale model depth, width, and resolution, making it parameter-efficient while preserving representational capacity. No layers are frozen, so all weights are updated during fine-tuning. We replaced the final classification layer with a fully-connected layer mapping the 1,280-dimensional feature vector to predictions for our 36 classes.

Our third model applies transfer learning using ResNet-18, an 18-layer architecture with residual connections that add block inputs to their outputs, mitigating the vanishing gradient problem. Unlike EfficientNet-B0, we adopted a partial fine-tuning strategy where all layers are initially frozen, then layer4 and the fully-connected layer are unfrozen for training. This preserves the general low-level features learned by early layers while allowing deeper layers to adapt to our dataset, and reduces the number of

trainable parameters as an implicit regularizer. As with EfficientNet, we replaced the final layer to output predictions for our 36 classes.

Experiments

To set up our experiments for model comparison, we split the dataset into train, validation, and test sets. We initially tried a standard random split (70% training, 10% validation, 20% test); however, we observed near-perfect model performance, suggesting a potential issue with data leakage. After looking into it further, we found that the dataset is organized by volunteer, with each volunteer contributing 100 highly similar images per class. A random split therefore distributed images from the same volunteer across all three sets, allowing the model to be evaluated on images nearly identical to those seen during training, which is a form of subject-level data leakage that artificially inflated performance.

To address this, we split the data by volunteer instead: volunteers 0-6 were assigned to training, volunteer 7 to validation, and volunteers 8-9 to the test set. This yielded 25,200 training images, 3,600 validation images, and 7,200 test images. Ideally, a dataset would contain a larger number of volunteers with greater variation between samples to make sure each image is meaningfully distinct. However, such datasets are limited, and many existing ASL image datasets exhibit strong intra-subject similarity, making careful data splitting essential. By ensuring that no volunteer appears in more than one split, our approach forced the model to generalize to entirely unseen individuals. This provided a more realistic evaluation of model performance, as it better reflects real-world deployment scenarios where the model must recognize gestures from new users.

Before model training and after splitting, we applied data augmentation to the training set to increase diversity and improve generalization. To augment the data, we performed random rotation (± 15 degrees), random translation (up to 10%), color jitter (brightness/contrast $\pm 30\%$, saturation $\pm 20\%$), Gaussian blur, random grayscale (10%), random perspective transformation, and random horizontal flip.

After training all three models, EfficientNet-B0 had the highest training, validation, and test accuracy. The baseline CNN had the lowest test loss of the three models. ResNet-18 achieved the second highest test accuracy, while the baseline CNN had the lowest. Table 1 summarizes the evaluation metrics across all models.

Model	Train Accuracy	Best Validation Accuracy	Test Accuracy	Test Loss
Model 1: Scratch CNN	91.05	83.89	77.93	0.5454
Model 2: EfficientNet	99.95	88.97	84.36	0.6059
Model 3: ResNet18	99.92	85.03	81.19	0.6355

Table 1: Model Comparison Summary

Figure 1 illustrates cross-entropy loss for the baseline CNN. Loss decreases drastically in the first several epochs, then more steadily, reaching its lowest validation loss at epoch 11, after which validation loss starts to rise which indicates overfitting.

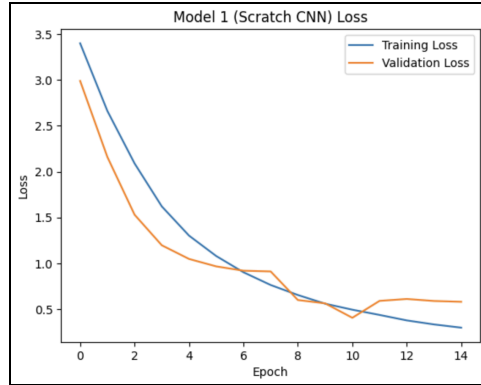


Figure 1: Model 1 (Scratch CNN) Loss Curves

Figure 2 shows the loss curves for EfficientNet-B0. Training loss drops rapidly toward zero within the first few epochs, signaling that the model learns the patterns of the training data very quickly. The validation loss begins around 0.4 and fluctuates without consistent improvement throughout training. This divergence indicates overfitting because despite fitting the training distribution closely, the model does not consistently improve on unseen data.

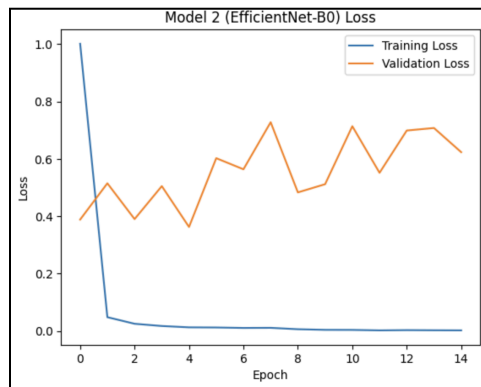


Figure 2: Model 2 (EfficientNet-B0) Loss Curves

Figure 3 shows the loss curves for the ResNet-18 model. As with EfficientNet-B0, training loss quickly drops towards 0 within the first few epochs, again indicating rapid learning. The validation loss begins higher (around 0.75), decreases briefly, sharply increases around epoch 4, and then fluctuates at higher levels through epochs 5-10. This behavior indicates overfitting, and of the three models, ResNet-18 exhibits the highest validation loss.

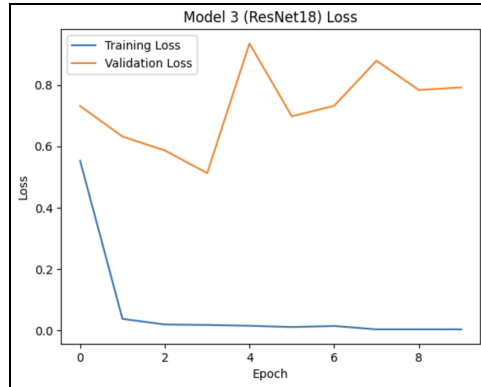


Figure 3: Model 3 (ResNet-18) Loss Curves

Confusion matrix analysis showed that most misclassifications occurred among visually similar gestures, particularly those distinguished by subtle finger orientations (e.g., S vs. N). This suggests the models are making systematic rather than random errors, struggling with fine-grained visual differences that could be improved with better feature learning or higher-resolution inputs.

Conclusions

This project demonstrates that deep learning models can classify ASL hand gestures with meaningful accuracy, while also surfacing challenges that stand between a research prototype and a practically useful system. Across the three architectures (a custom CNN trained from scratch, EfficientNet-B0 with full fine-tuning, and a partially frozen ResNet-18), EfficientNet-B0 achieved the highest test accuracy at 84.36%, while the baseline CNN produced the lowest test loss and exhibited the most stable training behavior of the three models. The gap between these results reflects a broader tradeoff we observed throughout: transfer learning methods tended to reach higher peak accuracy, but were more prone to overfitting, while the simpler architecture generalized more steadily despite its lower ceiling.

To demonstrate our best model in practice, we built a Gradio web application with two features. The first generates a fingerspelling video from any text input by displaying a sequence of dataset images for each letter or digit in the submission. The second takes a user-uploaded image of an ASL hand sign and returns the top three predicted classes with confidence scores using our EfficientNet-B0 classifier. In practice, the classifier performed well on visually distinct signs, but struggled with similar-looking signs like P and Q, which are distinguished by subtle differences in finger orientation. This limitation points to the need for even more diverse training data and potentially finer-grained model attention to hand geometry.

The most instructive lesson of this project came from an early mistake. Our initial random train/test split produced near-perfect model accuracy, which did not indicate success. Instead, it revealed data leakage between volunteers because the model had memorized subject-specific features rather than learning the signs themselves. We switched to a volunteer-stratified split which produced much more honest results. This is a good example of why evaluation methodology matters as much as model architecture because a model that achieves perfect accuracy on a leaky test set is useless in any real application. Looking forward, incorporating early stopping, a more diverse dataset, and landmark-based preprocessing could all meaningfully improve generalization and bring this kind of system closer to real-world deployment.

References

- Alsharif, B., Altaher, A. S., Altaher, A., Ilyas, M., & Alalwany, E. (2023). Deep Learning Technology to Recognize American Sign Language Alphabet. *Sensors*, 23(18), 7970. <https://doi.org/10.3390/s23187970>
- James, T. G., McKee, M. M., Sullivan, M. K., Ashton, G., Hardy, S. J., Santiago, Y., Phillips, D. G., & Cheong, J. (2021). Community-Engaged Needs Assessment of Deaf American Sign Language Users in Florida, 2018. *Public Health Reports*, 137(4), 730–738. <https://doi.org/10.1177/00333549211026782>
- Kasapbaşı, A., Elbushra, A. E. A., Al-hardanee, O., & Yilmaz, A. (2022). DeepASLR: A CNN based human computer interface for American Sign Language recognition for hearing-impaired individuals. *Computer Methods and Programs in Biomedicine Update*, 2, 100048. <https://doi.org/10.1016/j.cmpbup.2021.100048>
- Pranto, Md. F. I., Islam, Md. R., Akbor, Md. A., Ghosh, N., Alam, Md. R., Chaki, S., & Islam, Md. M. (2026). A comprehensive image dataset of American Sign Language hand gestures. *Data in Brief*, 65, 112492. <https://doi.org/10.1016/j.dib.2026.112492>
- Zulu, T., Heap, M., & Sinanovic, E. (2017). The cost and utilisation patterns of a pilot sign language interpreter service for primary health care services in South Africa. *PLOS ONE*, 12(12), e0189983. <https://doi.org/10.1371/journal.pone.0189983>

AI Use Statement We used AI tools (Claude, ChatGPT) to support our workflow by helping us work through challenges, clarify complex concepts, and refine portions of our code. These tools served as a guide when we encountered obstacles and needed additional direction.